

# CREACIÓN DEL PROYECTO FRONTEND



## 🔧 Modificar una Persona Existente en Nuestra Aplicación Angular

### 🌟 Introducción

En esta guía aprenderás a implementar la funcionalidad para modificar una persona existente en tu aplicación Angular. Aprovecharemos el componente ya creado para agregar personas y lo reutilizaremos con una lógica condicional para identificar si se trata de un nuevo registro o una modificación. También configuraremos las rutas necesarias, ajustaremos el enlace en la lista de personas y actualizaremos el arreglo local de personas después de guardar los cambios. ¡Vamos paso a paso! 🚀

---

### 🔧 Paso 1: Agregar el parámetro `idPersona` a las rutas

### Ruta del archivo:

src/app/app.routes.ts

### Descripción:

Agregaremos un nuevo path con parámetro `idPersona` para reutilizar el formulario en el caso de modificar una persona.

### Código:

```
import { Routes } from '@angular/router';
import { PersonasComponent } from '../personas/personas.component';
import { FormularioComponent } from '../formulario/formulario.component';

export const routes: Routes = [
  { path: '', component: PersonasComponent },
  { path: 'personas', component: PersonasComponent, children: [
    { path: 'agregar', component: FormularioComponent },
    { path: ':idPersona', component: FormularioComponent }
  ] }
];
```

### Explicación:

Este nuevo path permite que el componente `FormularioComponent` se cargue cuando se proporciona un parámetro `idPersona`, indicando que estamos en modo edición. 



## Paso 2: Hacer clic en un nombre como enlace para editar

### Ruta del archivo:

src/app/personas/personas.component.html

### Descripción:

Convertiremos los nombres en la lista de personas en enlaces que redirigen al formulario de edición con su respectivo `idPersona`.

### Código:

```
<div style="text-align: center">
  <button style="cursor: pointer" (click)="irAgregar()">+</button>
</div>
```

```
<div class="box">
  @for (persona of personas; track persona.idPersona) {
    <div class="item">
      <ul>
        <li>
          <a [routerLink]="['/personas', persona.idPersona]">
            {{persona.nombre}}
          </a>
        </li>
      </ul>
    </div>
  }
</div>
<!-- incrusta el componente de formulario -->
<router-outlet></router-outlet>
```

## Explicación:

Usamos `routerLink` con el ID de la persona para que, al hacer clic en su nombre, nos lleve al formulario ya precargado para editarla. 📄



## Paso 3: Detectar si se está modificando una persona



### Ruta del archivo:

src/app/formulario/formulario.component.ts

## Descripción:

Dentro del ciclo de vida `ngOnInit`, identificaremos si se recibió un `idPersona`. Si es así, obtenemos la persona y precargamos su nombre en el formulario.

Además, modificaremos la lógica del botón “Guardar” para diferenciar si se debe agregar o modificar una persona, en el método `onGuardarPersona`.

## Código:

```
import { Component } from '@angular/core';
import { PersonaService } from '../persona-service';
import { ActivatedRoute, Router } from '@angular/router';
import { Persona } from '../persona.model';
import { FormsModule } from '@angular/forms';
```

```
@Component({
  selector: 'app-formulario',
  imports: [FormsModule],
  templateUrl: './formulario.component.html',
  styles: ``
})
export class FormularioComponent {
  idPersona?: number;
  nombreInput?: string;

  constructor(private personaService: PersonaService,
    private router: Router,
    private route: ActivatedRoute
  ) { }

  ngOnInit() {
    this.idPersona = Number(this.route.snapshot.params['idPersona']);
    console.log('recuperamos el parametro idPersona:' + this.idPersona);
    if (this.idPersona != null) {
      const persona = this.personaService.encontrarPersona(this.idPersona);
      if (persona != null) {
        this.nombreInput = persona.nombre;
      }
    }
  }

  onGuardarPersona() {
    const personaAGuardar = new Persona(this.idPersona ?? 0, this.nombreInput ?? '');
    if (this.idPersona != null) {
      this.personaService.modificarPersona(this.idPersona, personaAGuardar);
    }
    else {
      this.personaService.agregarPersona(personaAGuardar);
    }
    this.router.navigate(['personas']);
  }
}
```

## Explicación:

Recuperamos el `idPersona` desde la URL en el método `ngOnInit`. Si existe, obtenemos la persona y cargamos su nombre en el campo del formulario gracias al two-way binding. 🌸

Se construye el objeto `Persona` con el ID recibido (si lo hay) en el método `onGuardarPersona`. Si existe el ID, se modifica; si no, se agrega. Finalmente se regresa a la lista. 🚀

---

## 🔁 Paso 4: Actualizar el arreglo local tras modificar

### 📄 Ruta del archivo:

`src/app/persona.service.ts`

### Descripción:

Actualizaremos el arreglo de personas en memoria, dentro del método `modificarPersona` para que refleje los cambios sin necesidad de recargar.

### Código:

```
import { Injectable } from '@angular/core';
import { Persona } from '../persona.model';
import { DataService } from '../data-service';

@Injectable({
  providedIn: 'root'
})
export class PersonaService {

  personas: Persona[] = [];

  constructor(private dataService: DataService) {}

  setPersonas(personas: Persona[]) {
    this.personas = personas;
  }

  obtenerPersonas() {
    return this.dataService.cargarPersonas();
  }

  agregarPersona(persona: Persona) {
    console.log('persona a agregar:' + persona.nombre);
    this.dataService.agregarPersona(persona)
      .subscribe({
        next: (persona: Persona) => {
          console.log('se agrega al arreglo la persona recién insertada suscriber:' + persona.idPersona);
        }
      });
  }
}
```

```
        this.personas.push(persona);
    },
    error: (error) => {
        console.error('Error al agregar persona:', error);
    }
});
}

encontrarPersona(id: number){
    const persona = this.personas.find(p => p.idPersona == id);
    console.log('persona encontrada:' + persona?.idPersona + ' ' + persona?.nombre);
    return persona;
}

modificarPersona(id: number, persona: Persona) {
    console.log('persona a modificar:' + persona.idPersona);
    // se actualiza el objeto persona del arreglo
    const personaModificadaLocal = this.personas.find(persona => persona.idPersona == id);
    if (personaModificadaLocal) {
        personaModificadaLocal.idPersona = persona.idPersona;
        personaModificadaLocal.nombre = persona.nombre;
    }
    // Se guarda la persona en la BD
    this.dataService.modificarPersona(id, persona);
}

eliminarPersona(id: number) {
    console.log('eliminar persona con id:' + id);
    const index = this.personas.findIndex(p => p.idPersona == id);
    this.personas.splice(index, 1);
    this.dataService.eliminarPersona(id);
}
}
```

## Explicación:

Se busca la persona en el arreglo local y se actualiza su nombre. Luego, se envía la modificación al backend por medio del `dataService`. 



## Paso 5: Probar que la modificación funcione

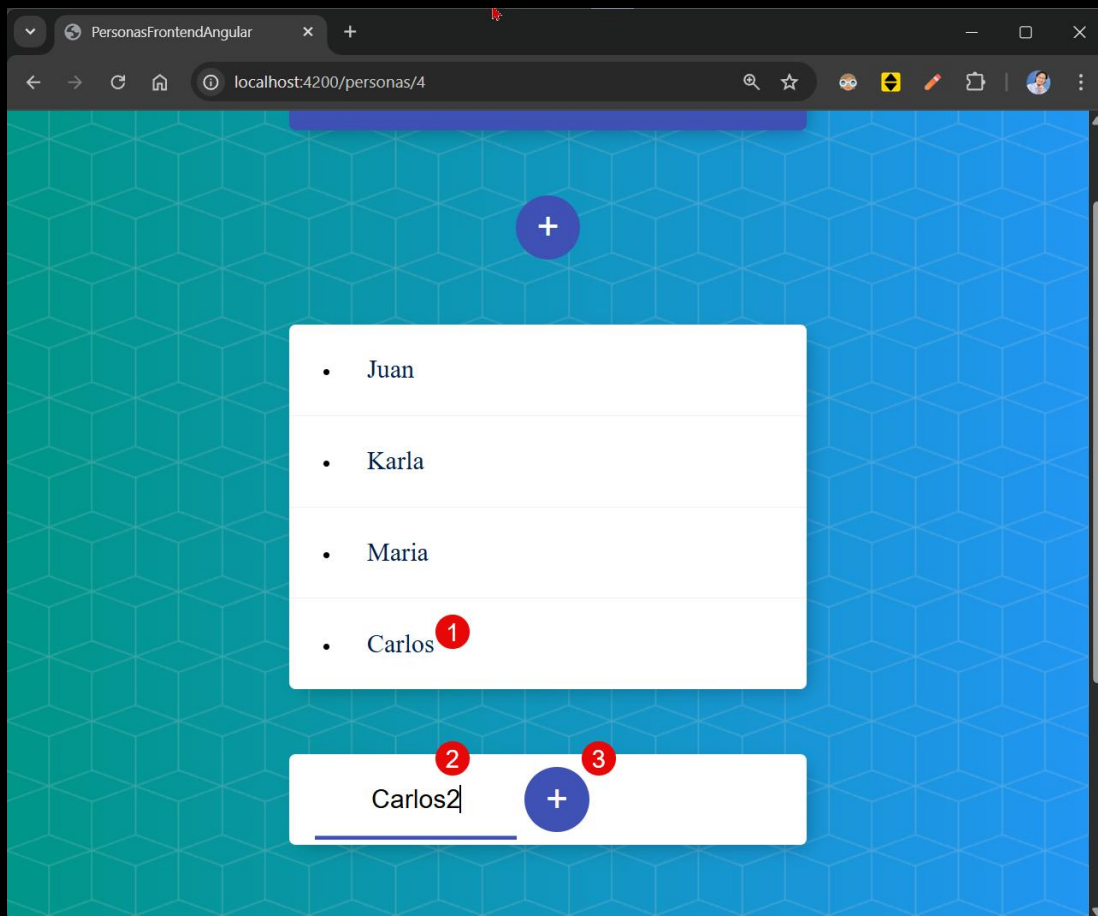
## Descripción:

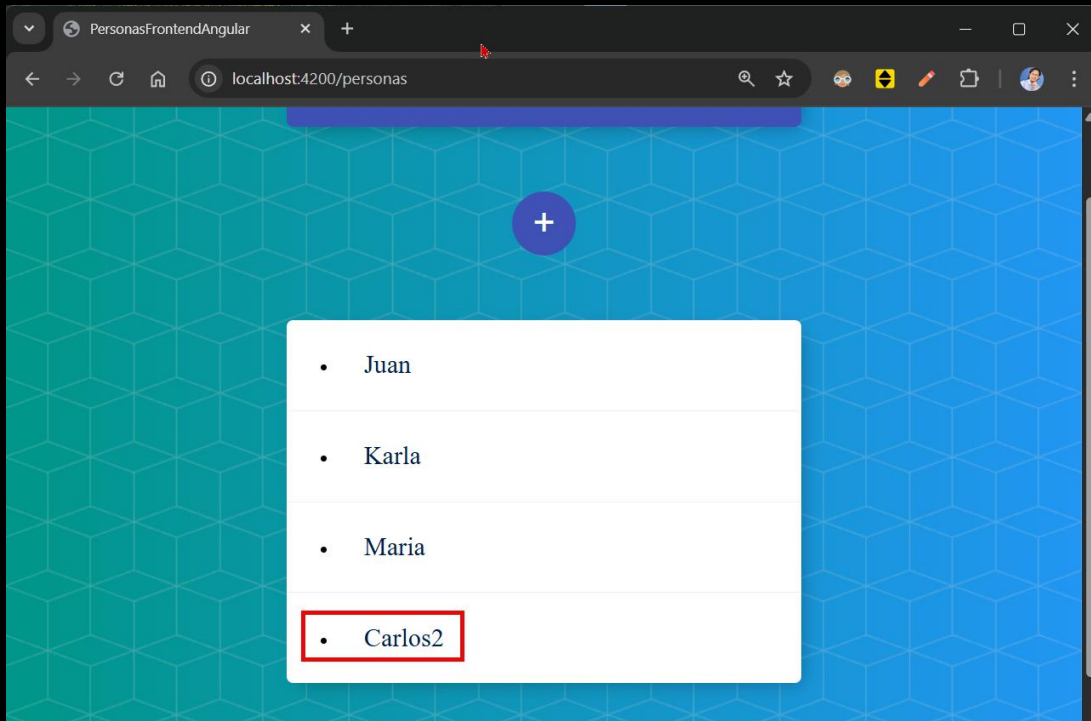
Verificamos que el nombre de la persona se cargue correctamente al editar, y que se actualice en la lista tras hacer clic en "+".

## Pasos:

- Abre la app y haz clic en el nombre de una persona.
- Cambia su nombre.
- Haz clic en "+".
- Verifica que el nombre actualizado aparezca en la lista.

🎉 ¡Ya puedes editar personas correctamente!





## Conclusión

En esta guía aprendiste cómo modificar registros existentes reutilizando componentes en Angular. Aprendiste a detectar parámetros en rutas, cargar datos dinámicamente y actualizar el estado local de la aplicación. Esta funcionalidad es esencial en cualquier CRUD moderno 🧑💻.

Sigue adelante con tu aprendizaje 🚀, ¡el esfuerzo vale la pena!

¡Saludos! 🙌

Ing. Marcela Gamiño e Ing. Ubaldo Acosta

Fundadores de [GlobalMentoring.com.mx](https://www.globalmentoring.com.mx)